```
#!/usr/bin/env bash
#
# vid2gif.sh — convert a video file to an
optimized animated GIF
#
# Usage:
#   ./vid2gif.sh -i input.mp4 [-o
output.gif] [-f fps] [-w width] [-s start]
[-t duration] [-c]
#
# Options:
#   -i  Input video file (required)
#   -o  Output GIF path (default:
<input_basename>.gif)
#   -f  Frames per second (default: 15)
#   -w  Output width in px, height auto-
scaled keeping aspect ratio (default: 800)
#   -s  Start time, e.g. 00:00:05 or 5
(seconds) — optional, trims from here
#   -t  Duration to encode, e.g. 00:00:10
or 10 (seconds) — optional
#   -c  Compress the final GIF with
gifsicle -O3 (requires gifsicle)
#   -h  Show this help
#
# Requires: ffmpeg (and optionally gifsicle
for -c)
#
# Example:
#   ./vid2gif.sh -i demo.mp4 -o demo.gif -f
```

```
12 -w 900 -s 3 -t 8 -c

set -euo pipefail

INPUT=""
OUTPUT=""
FPS=15
WIDTH=800
START=""
DURATION=""
COMPRESS=0

usage() {
    grep '^#' "$0" | sed -n '2,25p' | sed
's/^#//; s/^ //'
    exit 1
}

while getopts ":i:o:f:w:s:t:ch" opt; do
    case "$opt" in
        i) INPUT="$OPTARG" ;;
        o) OUTPUT="$OPTARG" ;;
        f) FPS="$OPTARG" ;;
        w) WIDTH="$OPTARG" ;;
        s) START="$OPTARG" ;;
        t) DURATION="$OPTARG" ;;
        c) COMPRESS=1 ;;
        h) usage ;;
        \?) echo "Unknown option: -$OPTARG"
>&2; usage ;;
        :)  echo "Option -$OPTARG requires
an argument." >&2; usage ;;
    esac
done
```

```
if [[ -z "$INPUT" ]]; then
    echo "Error: input file is required (-
i)" >&2
    usage
fi

if [[ ! -f "$INPUT" ]]; then
    echo "Error: input file '$INPUT' not
found" >&2
    exit 1
fi

if ! command -v ffmpeg >/dev/null 2>&1;
then
    echo "Error: ffmpeg is not installed or
not on PATH" >&2
    exit 1
fi

if [[ -z "$OUTPUT" ]]; then
    BASE="${INPUT%.*}"
    OUTPUT="${BASE}.gif"
fi

# Build optional seek/duration args
SEEK_ARGS=()
if [[ -n "$START" ]]; then
    SEEK_ARGS+=(-ss "$START")
fi
DURATION_ARGS=()
if [[ -n "$DURATION" ]]; then
    DURATION_ARGS+=(-t "$DURATION")
fi
```

```
WORKDIR="$(mktemp -d)"
PALETTE="${WORKDIR}/palette.png"

cleanup() {
    rm -rf "$WORKDIR"
}
trap cleanup EXIT

echo "==> Generating palette..."
ffmpeg -y "${SEEK_ARGS[@]}" -i "$INPUT"
"${DURATION_ARGS[@]}" \
    -vf
"fps=${FPS},scale=${WIDTH}:-1:flags=lanczos
,palettegen" \
    "$PALETTE" -loglevel error -stats

echo "==> Rendering GIF..."
ffmpeg -y "${SEEK_ARGS[@]}" -i "$INPUT"
"${DURATION_ARGS[@]}" -i "$PALETTE" \
    -filter_complex
"fps=${FPS},scale=${WIDTH}:-1:flags=lanczos
[x];[x][1:v]paletteuse" \
    "$OUTPUT" -loglevel error -stats

if [[ "$COMPRESS" -eq 1 ]]; then
    if command -v gifsicle >/dev/null 2>&1;
then
        echo "==> Compressing with
gifsicle..."

TMP_OPT="${OUTPUT%.gif}_optimized.gif"
        gifsicle -O3 "$OUTPUT" -o
"$TMP_OPT"
```

```
        mv "$TMP_OPT" "$OUTPUT"
    else
        echo "Warning: gifsicle not found,
skipping compression (-c ignored)" >&2
    fi
fi

SIZE=$(du -h "$OUTPUT" | cut -f1)
echo "==> Done: $OUTPUT ($SIZE)"
```